Comparaison d'algorithmes de tri en RISC-V

Hubert Lestrat
Polytech-Sorbonne — MAIN3 Architecture des ordinateurs

23 avril 2026

Table des matières

1 Références **1**

2 Astuces **1**

3 Méthode de mesures **1**
3.1 Dans RARS 1

4 Résultats **2**

5 VRAC à travailler **2**
5.1 Temps (= nombre d'instructions) 2
5.2 Mémoire 3

1 Références

- https://www.laurentbloch.net/MySpip3/Mon-premier-programme-en-assembleur-RISC-V
- https://www.laurentbloch.net/MySpip3/Nouvel-episode-de-programmation-en-assembleur-RISC-
- https://www.laurentbloch.net/MySpip3/Assembleur-RISC-V-maintenant-les-tris
- https://fr.wikipedia.org/wiki/Tri_%C3%A0_bulles

2 Astuces

`rars test.asm` ⇒ compile et lance le programme en console.

Attention, lorsqu'on « include » un fichier, les étiquettes doivent rester uniques. Le mieux est de les préfixer avec le nom de la fonction.

Générer un tableau aléatoire de 20 entrées (pour `tab.asm`) :

```
python3 -c "import random; random.seed(); \
  print(', '.join(str(random.randint(-99, 99)) for _ in range(20)))"
```

3 Méthode de mesures

3.1 Dans RARS

- Ouvrir le fichier `.asm` à tester.

- `Tools` → `Instruction Counter`, puis « Connect to program ».
- `Settings` → `Hexadecimal Values`.
- `Settings` → `Hexadecimal Addresses`.

1. **Assemble** (F3).
2. Clic sur **Reset** dans la fenêtre Instruction Counter (important : vider entre deux tests).
3. **Run** (F5) jusqu'à la fin.
4. Relever les chiffres.

Exécution. Dans `Instruction Counter` :

- Ligne *Instructions so far* → métrique principale.
- Décomposition R / I / S / B / U / J → utile pour distinguer calcul pur vs accès mémoire (les S-type = stores, les loads sont des I-type).

Mémoire — lecture directe dans RARS.

- **Text Segment** : scroller jusqu'à la dernière instruction de la fonction, noter l'adresse du label d'entrée et l'adresse juste après `ret`. Différence = taille du code en octets.
- **Data Segment** : idem pour `tab` et les constantes. On y voit le tableau avant/après le tri, utile pour vérifier.
- **Pile** : regarder `sp` dans la fenêtre *Registers* avant l'appel, puis mettre un breakpoint sur la 1re instruction de la fonction pour voir `sp` après le prologue. La différence = octets consommés sur la pile par appel. Pour un algo itératif comme le tri à bulles, elle ne grandit jamais au-delà.

4 Résultats

Les appels des fonctions communes sont considérés négligeables. Par exemple ***`print_tab`*** *est appelée de la même manière sur tous les algorithmes.*

Algorithme	**Instr. tot.**	**R**	**I**	**S**	**B**	**U**	**J**	**Pile (o)**	**Récursif**	**Complexité**
Tri à bulles	**2 698**	709	1 097	235	322	48	287	20	non	$O(n^2)$
Tri par sélection										$O(n^2)$
Tri par insertion										$O(n^2)$
Tri rapide									oui	$O(n \log n)$ moy.
Tri fusion									oui	$O(n \log n)$
Tri par tas	**2 481**	925	830	144	385	48	149	16	non	$O(n \log n)$

Table 1: Comparaison des algorithmes de tri sur un tableau de 20 entiers.

Type de comparaison	Formule	Ce que ça dit
Ratio entre algos	$\text{total}_A/\text{total}_B$	« A est 2,3× plus lent que B »
Intensité mémoire	$(I+S)/\text{total}$	part des instructions qui touchent la RAM
Vérification théorique	total/n^2 ou $\text{total}/(n\log_2 n)$	le coefficient doit rester à peu près constant si l'on refait le test avec un autre n

5 VRAC à travailler

5.1 Temps (= nombre d'instructions)

Comparaison directe : la colonne *Total instr.* suffit. Plus petit = plus rapide.

Trois lectures utiles :

La dernière ligne est la plus intéressante pour un rapport d'archi : elle valide (ou non) la complexité théorique. Pour la bulle avec $n = 20$, le coût « pur tri » $\approx 2698 -$ print instructions, divisé par $20^2 = 400 \to$ on obtient une constante $\sim X$. En refaisant avec $n = 50$, on doit retomber sur un ordre $\sim X$.

5.2 Mémoire

Là, un seul chiffre ne suffit pas. Il faut distinguer deux composantes.

Composant	Comment mesurer dans RARS
Taille du code de l'algo	adresse de `ret` − adresse du label d'entrée (en octets)
Données auxiliaires allouées	regarder `.data` du fichier : tables de travail, buffers...

a. Mémoire statique (code + données). Pour la bulle : juste du code, 0 octet de données auxiliaires (tri in-place). Pour le tri fusion : on allouera un tableau temporaire $\to +4n$ octets de données.

b. Mémoire dynamique (pile) — c'est ça qui diffère vraiment. La bonne métrique n'est pas un octet absolu, c'est la fonction de n :

Algo type	Pile consommée	Pour $n = 20$	Pour $n = 1000$
Itératif (bulle, insertion, sélection)	constante (1 cadre)	20 o	20 o
Récursif équilibré (fusion, tas)	cadre $\times \log_2(n)$	~ 5 cadres	~ 10 cadres
Récursif dégénéré (quicksort pire)	cadre $\times n$	20 cadres	1000 cadres !

Dans RARS on lit :

- **Taille d'un cadre** → dans le prologue : `addi sp, sp, -N` → N octets.
- **Profondeur max** → registre `sp` pendant l'exécution (breakpoint à l'intérieur de la fonction récursive), ou instrumentation `sp_min` si besoin.

Résumé : ce qu'il faut présenter dans le rapport. Deux tableaux, pas un.

Tableau 1 — Temps

Algo	Total instr.	Ratio vs plus rapide	Coefficient total/n^2 ou /$(n \log n)$

Tableau 2 — Mémoire

Algo	Code (o)	Données aux. (o)	Pile/cadre (o)	Profondeur pile	Pile max (o)

La colonne *Pile max* (= cadre × profondeur) est la métrique clé pour comparer l'empreinte mémoire dynamique entre algos.

Conclusion qualitative attendue. On devrait retrouver dans les chiffres :

- **Bulle** : lent ($O(n^2)$) mais mémoire minimale (20 o fixes).
- **Fusion** : rapide ($O(n \log n)$) mais paye en mémoire ($4n$ o de buffer + pile $\log_2 n$).
- **Quicksort** : rapide en moyenne, mais pile variable selon les données → risqué sur gros n.